DISCRETE STRUCTURES

CMSC250

Chapter 5: Logic Gates and Binary Circuits

By Naren Ayinala

From truth tables to circuits and binary addition



Notation. We use $\sim p$ for negation (also written $\neg p$). Binary values 1 and 0 represent true and false. All circuit diagrams are schematic.

Contents

1 Why logical expressions become circuits **3**

2 The basic gates **3**

2.1 NOT, AND, and OR 3

2.2 XOR: exactly one input is true 3

2.3 NAND and NOR 4

3 Reading a circuit from left to right **4**

4 Constructing circuits from expressions **4**

5 Constructing expressions from truth tables **4**

6 Binary numbers and place values **5**

6.1 Binary addition and carrying 5

7 The half-adder **6**

8 The full-adder **6**

8.1 Why the full-adder equations work 7

9 Parallel adders: adding many bits **7**

9.1 How many adders are needed? 7

10 Common mistakes and how to avoid them **8**

11 Practice problems **8**

12 Detailed solutions **9**

1 Why logical expressions become circuits

A logical formula describes an output determined by inputs. A digital circuit does the same thing with physical signals: a high signal represents 1 and a low signal represents 0. For example, $p \wedge q$ produces 1 only when both inputs are 1. A circuit implementing it is called an **AND gate**.

The connection is useful because we can move in either direction: from a logical expression to a circuit, or from a circuit to an expression and truth table. Later, we will combine these gates to add binary numbers.

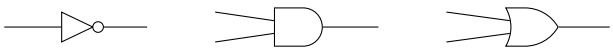
Key idea. A gate is a physical implementation of a Boolean operation. A circuit is a network of gates. To understand the circuit, follow how each intermediate signal is computed.

2 The basic gates

2.1 NOT, AND, and OR

The **NOT gate** has one input and reverses its truth value. The **AND** and **OR** gates have two inputs. In this course, OR is *inclusive*: it is true when one or both inputs are true.

p	q	$\sim p$	$p \wedge q$	$p \vee q$
1	1	0	1	1
1	0	0	0	1
0	1	1	0	1
0	0	1	0	0



NOT AND OR

The small circle (*bubble*) on a NOT gate indicates inversion. You may also see bubbles attached to the outputs of other gates to indicate negation.

2.2 XOR: exactly one input is true

The **exclusive OR** (XOR) operation, written $p \oplus q$, is true precisely when the inputs differ:

$$p \oplus q \;\equiv\; (p \wedge \sim q) \vee (\sim p \wedge q).$$

p	q	$p \oplus q$
1	1	0
1	0	1
0	1	1
0	0	0

Do not confuse $p \oplus q$ with $p \vee q$. The difference occurs when $p = q = 1$: inclusive OR gives 1, XOR gives 0.

2.3 NAND and NOR

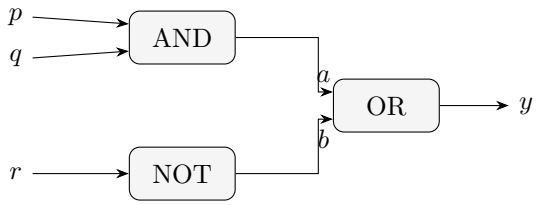
A **NAND** gate computes $\sim (p \wedge q)$; a **NOR** gate computes $\sim (p \vee q)$. They are the negations of AND and OR, respectively. For example, NAND produces 0 only when both inputs are 1.

3 Reading a circuit from left to right

A complicated circuit becomes manageable if we label each wire with an intermediate expression. Consider the output

$$y = (p \wedge q) \vee \sim r.$$

First compute $a = p \wedge q$ and $b = \sim r$. Then compute $y = a \vee b$.



Worked example: evaluating a circuit

Suppose $p = 1, q = 0, r = 0$. The AND gate gives $a = 1 \wedge 0 = 0$; the NOT gate gives $b = \sim 0 = 1$; the OR gate gives $y = 0 \vee 1 = 1$. This is much safer than trying to evaluate the entire expression mentally.

4 Constructing circuits from expressions

To implement $(p \vee q) \wedge \sim r$, draw an OR gate receiving p, q, a NOT gate receiving r, and an AND gate combining their outputs. Parentheses indicate which operation must happen first.

Worked example: implementing a three-variable formula

Build $f(p, q, r) = (p \wedge \sim q) \vee r$ using one NOT, one AND, and one OR gate:

1. Invert q to obtain $\sim q$.
2. AND p with $\sim q$ to obtain $t = p \wedge \sim q$.
3. OR t with r to obtain $f = t \vee r$.

Every gate corresponds to exactly one operation in the formula.

5 Constructing expressions from truth tables

Suppose we are given a truth table but no formula. One systematic method is **sum of products**: for each row with output 1, construct an AND term that is true on that row and false on every other row. OR those terms together.

For the input row $p = 1, q = 0, r = 1$, the corresponding term is $p\wedge \sim q \wedge r$. Every variable appears: use the variable itself when the row contains 1, and its negation when the row contains 0.

Worked example: building a formula from a truth table

Consider a function f that is 1 exactly on the rows $(p, q) = (1, 0)$ and $(0, 1)$. The row terms are $p\wedge \sim q$ and $\sim p \wedge q$. Therefore

$$f = (p\wedge \sim q) \vee (\sim p \wedge q) = p \oplus q.$$

The sum-of-products expression is always correct if the chosen rows are correct. Simplifying it is a separate step.

Key idea. When asked for a circuit from a truth table, first write an expression for the output, then translate each operation into gates. This avoids guessing the circuit shape.

6 Binary numbers and place values

A binary number uses only digits 0 and 1. Its digits represent powers of two. For example,

$$(10110)_2 = 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 22.$$

The subscript 2 means "base two." In decimal, the number 22 has place values 10^1 and 10^0; in binary, place values are powers of 2.

Worked example: converting decimal to binary

Convert 13 to binary. The largest power of two not exceeding 13 is 8; subtract to obtain 5. Then use 4 (remainder 1) and 1. Hence

$$13 = 8 + 4 + 1 = (1101)_2.$$

6.1 Binary addition and carrying

Binary addition uses four elementary cases:

$$0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 0 = 1, \quad 1 + 1 = (10)_2.$$

The last case is important: $1 + 1$ produces a *sum bit* of 0 and a *carry bit* of 1. When adding three bits, $1 + 1 + 1 = (11)_2$, so both sum and carry are 1.

Worked example: adding two binary numbers

$$\begin{array}{r} 1011 \\ +\,0110 \\ \hline 10001 \end{array} \qquad \text{since} \qquad 11 + 6 = 17.$$

Starting at the right: $1 + 0$ gives 1; $1 + 1$ gives 0 with carry 1; $0 + 1 + 1$ gives 0 with carry 1; $1 + 0 + 1$ gives 0 with carry 1, which becomes the leftmost bit.

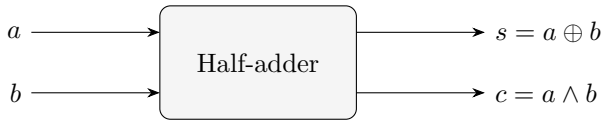
7 The half-adder

A **half-adder** adds two single bits a and b. It produces two outputs: a sum bit s and a carry-out bit c.

a	b	s	c
1	1	0	1
1	0	1	0
0	1	1	0
0	0	0	0

Compare the rows with the truth tables of XOR and AND:

$$\boxed{s = a \oplus b} \qquad \boxed{c = a \wedge b}.$$



A half-adder is insufficient for adding digits in the middle of a multi-bit addition because it cannot accept a carry from the previous column.

8 The full-adder

A **full-adder** adds three input bits: a, b, and an incoming carry c_{in}. It returns a sum bit s and an outgoing carry c_{out}.

a	b	c_{in}	s	c_{out}
1	1	1	1	1
1	1	0	0	1
1	0	1	0	1
1	0	0	1	0
0	1	1	0	1
0	1	0	1	0
0	0	1	1	0
0	0	0	0	0

The sum bit is 1 when an odd number of the inputs are 1:

$$\boxed{s = a \oplus b \oplus c_{\text{in}}}.$$

The carry-out bit is 1 when at least two of the three inputs are 1:

$$\boxed{c_{\text{out}} = (a \wedge b) \vee (a \wedge c_{\text{in}}) \vee (b \wedge c_{\text{in}})}.$$

An equivalent expression, convenient for building a circuit, is

$$c_{\text{out}} = (a \wedge b) \vee \big(c_{\text{in}} \wedge (a \oplus b)\big).$$

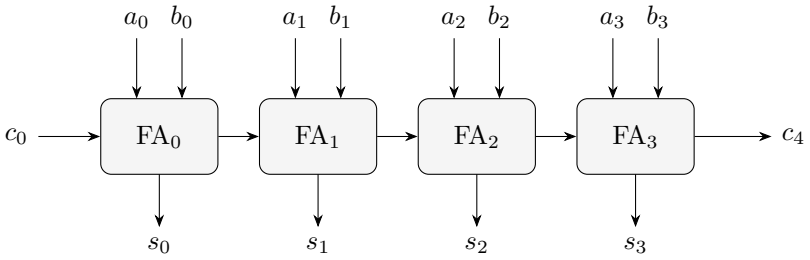
8.1 Why the full-adder equations work

Let $t = a \oplus b$. The sum of a and b has low bit t and carry $a \wedge b$. Now add c_{in} to t: the new low bit is $t \oplus c_{\text{in}}$, and an additional carry is $t \wedge c_{\text{in}}$. The two possible carries cannot both be 1 at once, so their OR gives the outgoing carry.

Key idea. A full-adder can be built from two half-adders and one OR gate. The first half-adder adds a, b; the second adds its sum to c_{in}; the OR combines the two carry signals.

9 Parallel adders: adding many bits

A multi-bit adder places one adder at each bit position. The outgoing carry from the current position becomes the incoming carry for the next position. This arrangement is often called a **ripple-carry adder**.



In this diagram, the bit positions increase from left to right for ease of following the carry path. Position 0 is the *least significant bit*. For ordinary addition, $c_0 = 0$.

9.1 How many adders are needed?

For two n-bit inputs, there are n bit positions. A straightforward design uses n full-adders (with $c_0 = 0$). An alternative uses one half-adder for the least significant position and $n - 1$ full-adders for the remaining positions. Both implement the same arithmetic.

For a **32-bit** adder, this means either 32 full-adders, or 1 half-adder and 31 full-adders. The final carry-out may be kept as an additional bit, discarded, or interpreted according to the number representation; it is not automatically the same as signed overflow.

Worked example: tracing carry propagation

Add $(1111)_2$ and $(0001)_2$. At bit 0, $1 + 1$ produces sum 0 and carry 1. At bits 1, 2, and 3, each column adds $1 + 0 + 1$, producing sum 0 and carry 1. The final carry becomes the fifth bit:

$$(1111)_2 + (0001)_2 = (10000)_2.$$

This shows why carry connections between positions are essential.

10 Common mistakes and how to avoid them

1. **Using OR when XOR is needed.** In a half-adder, $1 + 1$ gives sum 0, not 1; the carry is 1.
2. **Ignoring the incoming carry.** A half-adder cannot generally replace a full-adder at higher bit positions.
3. **Confusing output sum with carry.** The sum is the low-order bit; the carry represents the next power of two.
4. **Omitting input combinations.** A truth table with three independent inputs needs $2^3 = 8$ rows.
5. **Guessing a circuit from the output.** Label intermediate signals and derive the Boolean formula first.
6. **Mixing up bit order.** Bit 0 is the least significant position; carry flows toward more significant positions.
7. **Using ambiguous notation.** Label c_{in} and c_{out} separately, and use $\sim$ for negation.

11 Practice problems

Try each question before reading the solutions. Use 1 and 0 for truth values and descending binary row order in truth tables.

1. Construct the truth tables of NAND and NOR.

2. Construct the truth table of $f = (p \wedge \sim q) \vee r$.

3. Describe a gate-level implementation of $\sim (p \vee q) \wedge r$.

4. Find a sum-of-products expression for a two-input output that is 1 exactly when $p = q$.

5. Explain the difference between OR and XOR using the input $p = q = 1$.

6. Convert $(101101)_2$ to decimal.

7. Convert 27 to binary, showing the powers of two used.

8. Add $(1011)_2 + (1101)_2$, showing carries.

9. Give the sum and carry for a half-adder with $a = b = 1$.

10. For a full-adder with $(a, b, c_{\text{in}}) = (1, 0, 1)$, find both outputs and explain.

11. Derive the full-adder sum expression using XOR.

12. Explain why the carry-out is 1 when at least two of the full-adder inputs are 1.

13. How many half-adders and full-adders are used in a 16-bit ripple-carry adder with a half-adder at bit 0?

14. How many full-adders are used in a 32-bit adder if every bit position uses a full-adder?

15. Describe the output of adding $(11111111)_2$ and $(00000001)_2$ when the final carry is retained.

12 Detailed solutions

1.

p	q	$\sim(p \wedge q)$	$\sim(p \vee q)$
1	1	0	0
1	0	1	0
0	1	1	0
0	0	1	1

.

2.

p	q	r	f
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	1
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	0

. When $r = 1$, the output is automatically 1. When $r = 0$, it equals $p \wedge \sim q$.

3. OR p, q; negate the OR output; AND that result with r. The final expression is $\sim(p \vee q) \wedge r$.

4. The output is 1 on rows $(1, 1)$ and $(0, 0)$. Therefore $f = (p \wedge q) \vee (\sim p \wedge \sim q)$, also called XNOR.

5. $p \vee q = 1$ while $p \oplus q = 0$ when $p = q = 1$. OR means at least one input is true; XOR means exactly one is true.

6. $32 + 8 + 4 + 1 = 45$.

7. $27 = 16 + 8 + 2 + 1$, so $27 = (11011)_2$.

8. $11 + 13 = 24$, so $(1011)_2 + (1101)_2 = (11000)_2$. From right to left: $1 + 1 = 0$ carry 1; $1 + 0 + 1 = 0$ carry 1; $0 + 1 + 1 = 0$ carry 1; $1 + 1 + 1 = 1$ carry 1, giving the leading 1.

9. $s = 1 \oplus 1 = 0$, $c = 1 \wedge 1 = 1$.

10. $1 + 0 + 1 = 2 = (10)_2$, so $s = 0$ and $c_{\text{out}} = 1$.

11. First XOR a, b to obtain $t = a \oplus b$. XOR t with c_{in} to obtain $s = (a \oplus b) \oplus c_{\text{in}}$.

12. The sum of three input bits is $0, 1, 2$, or 3. Values 2 and 3 have a carry bit of 1, and these occur exactly when at least two inputs equal 1.

13. One half-adder at bit 0 and 15 full-adders at bits 1 through 15.

14. 32 full-adders, one for each bit position.

15. The first eight sum bits are 0 and the carry-out is 1, giving $(100000000)_2 = 256$.